Your Data, Your Rules: Extending the Decentralized Web

Jacob Hoffman
web10 Inc.
jacob@web10.app

ABSTRACT

What makes your data worth owning is the graph around it: who follows you, who belongs to your groups, who you have allowed to read what. The content is portable in principle; the social graph—the follows, the groups, the audience—and the rules that go with them are the part no platform lets you keep. **web10** is a deployed, user-addressed data platform that closes that split: the access rules are your own data—grants are rows in the very store they protect—and a second application reads the first's data with per-service grants, finer than OAuth 2.0. A new read pattern is a new query, not a new backend. Your audience travels with you: an address is a domain you own, a node verifies but cannot control, and public subscriptions outlive a provider change. The platform takes no cut—creators keep 100% of their monetization, and nodes compete for them on terms, not leverage. At the bottom, ClickHouse at scale: at 1,000 followed creators, the cost lives in the access layer, not the database (7.64 s to 363 ms). When the data leaves a node, the rules leave with it.

1 THE SPLIT EVERY PLATFORM INHERITS

What makes your data worth owning is not the content. It is the graph around it: who follows you, who belongs to your groups, who you have allowed to read what, and what each of them can do there. The content is portable in principle—a post is a post—but the social graph—the follows, the groups, the audience—and the rules that go with it are the part no platform lets the user keep.

The systems the reader knows as "the decentralized web" each moved a piece of that. Solid [13] decentralized the *store*: one pod per user, the data in the user's hands, encrypted and pod-bound. Mastodon and Bluesky decentralized the *transport*: federated content between servers, and on Bluesky the choice of relays and app views with it—user agency and provider switching [8]. Every one of them left the access layer in the platform: who follows whom, who may read what, who belongs where, and what a second application may do with the first application's data are still decided by backends and object models the platform owns [9]. The content travels between their systems; the permissions do not.

web10 closes the split. It is a deployed, user-addressed data platform: the node that hosts it is completely generic—read and write on data, and configuration for identity and access, executed every time the data is used—and the access rules are the user's own data, in the very store they protect. A social application, a snake game, and a marketplace run on the node and interoperate on shared data through the same boundary: the game and the marketplace read the social application's `profile` documents with no node change and no API agreement between the applications (§5).

The contribution is a combination, and no part of it is new: user-owned storage, user-managed grants, and application-written queries over authorized relations are each established ideas (§10). What no prior system we compare carries is the conjunction—all three, on one shared columnar store, with the access decision made on the input before any data moves. Five properties, and this paper closes on each:

(1) **User-owned, portable data, all formats.** A post, a profile, a game score, and a marketplace listing are documents in the same store, differing only in the service string and the JSON shape the application agreed to use (§3).
(2) **Fine-grained, user-managed access.** A grant is a row: principal × group × service × operation; publicness is a grant, not a flag; the end user runs the IAM operations a cloud operator would express as policies, over their own data (§2).
(3) **Cross-node-addressable access.** A principal is `provider/username`, globally unique across the federation; a foreign token is verified, not registered—one cached `/certify` answer, the same access check on every read (§6).
(4) **Composable, app-defined queries.** The node has never learned a social concept—no posts table, no feed endpoint, no followers construct—and the query boundary keeps the shared store sound: the reader's readable set is compiled into the query before any data moves (§4).
(5) **A user-addressed network without federation.** Sovereign nodes, client-mediated reads, no central authority, no shared database; an address is a domain the user owns, and the audience travels with it when the account moves (§6).

The evidence is connected, bounded, and named (§9): a game and a marketplace reusing the social application's data through a real browser's consent and revocation; a grant that crosses a node boundary and is revoked at the owner's node; a public subscription that outlives its provider; and a measurement that found where the access layer's cost lives—7.64 s to 363 ms at 1,000 followed creators, with the one-group counterweight in the same trial.

Figure 1 draws the system the paper walks through. The browser frame holds the application's surfaces and the six data-layer modules a bespoke API would normally carry: feed and ranking (the app's own queries: joins, scoring, cursors), group discovery (the social graph as group rows the app reads and writes), faces and avatars, media resolution, request orchestration (batching, dedup, optimistic writes, retry), and acting identity (scoped, expiring tokens—what acts as whom)—plus the SDK. The node frame is thin and generic: authentication, the read gate, the query engine, and media (signed URLs to object storage; the render pipeline to HLS renditions). The stores are drawn with their real names: ClickHouse's `documents` and group tables, and the object store's media keys. The messages panel shows the payloads that move: `sql + token` down; the compiled query—not the caller's string—and rows with signed URLs back; then the signed URL,

the GET, and the bytes. §2 states what the node is and why the rules are the user's data; §3 the store at the bottom and the fork Solid did not take; §4 the query boundary; §5 how an extension costs a role and a policy; §6 the address, the handshake, and the domain; §7 where the money flows; §8 what running it taught; §9 what was measured; §10 where the combination sits against the systems it combines; §11 closes.

2 THE NODE STAYS GENERIC; THE RULES ARE YOUR DATA

The node that hosts web10 knows one thing, and it is not a social concept. It gives an application read and write on the data, and it gives it configuration for identity and access—and it executes that configuration every time the data is used. The platform API has no `posts` table, no feed endpoint, no followers construct: the harder the killer app pushes for bespoke surface (§8), the harder the node resists, because the next application—a notes app, a music app, a shop—defines identity differently or has no followers concept at all. The test that keeps the line: *would a notes app, a music app, or a shop use this endpoint?* Four such surfaces were built and removed from this node; the list is in §8.

What the node is not generic about is the access model—because the access model belongs to the user. A grant is a *row*: a principal, a group, and a role, where a role is a per-service permission map—a bundle named by the application that says `posts: readAll, create, updateOwn` and `comments: readAll`. Two rows carry the classes `anyone` and `authenticated`; a public group is a group that holds an `anyone` reader row. There is no visibility column, because publicness *is* a grant: "make my group public" is the act of writing that row, and "private" is the absence of it. One mechanism, and it composes—a group can be invite-only to *join* and still publicly *readable*, which is exactly the discovery board.

The operations this model exposes are the operations a cloud operator would express as policies—principal, service, operation, grant—but the user who expresses them is the end user, over their own data. The user revokes an app without leaving a group, leaves a group without touching an app grant, and rewrites their own publicness the way the platform's IAM owner would: as data, in the store, through the same API the applications use. A revoked grant is a tombstone; the checks that follow read the latest row.

That last sentence is the whole argument's forcing move. Because a grant is a row, the set of groups a reader may see—their readable set—is computable *before* any query runs, from data the user owns, in the same store the query will touch. A wall can be built on that; a filter cannot. §4 builds it.

Media moves the same way it is checked: structured documents reference objects in the node's object storage, and the node issues short-lived presigned URLs—and, for video, signed HLS manifests and segment capabilities—after the document's own access check, so the bytes have the document's authority and its expiry.

3 ONE STORE AT THE BOTTOM: CLICKHOUSE, AND THE FORK SOLID DID NOT TAKE

At the bottom of the system is ClickHouse [14]—an open-source columnar engine built for real-time analytics over billions of rows, in a way that scales. The platform's answer to "where does a user's data live?" is deliberately unglamorous: on each node, the user's content and the platform's access metadata live in *one* columnar database. The user namespace is logical isolation within that table—not a per-user database, not a physical collection; updates insert a newer version, deletes insert tombstones, and reads select the latest row and only then drop the deleted ones.

The multi-tenant literature places its tenants at the *organization*: shared tables carrying an organization identifier [15], schema-mapping techniques [1], and the shared-versus-separate spectrum [7]. web10 takes the spectrum to its limit: one logical tenant per *end user*, in a shared columnar store. Physical isolation was never what the access model needed. The isolation a reader requires—"show me only what I may see"—is a property of the *query*, and §4 supplies it at the query boundary.

web10 deliberately trusts the node operator with readable content: search, ranking, moderation, and audit operate on plaintext; the node is readable by design, and operator-blindness is not a goal. The stance is the opposite of the familiar decentralization bet, and it is worth stating plainly because the nearest prior work takes it. Solid [13] keeps its data encrypted and pod-bound for ownership. web10 makes the opposite bet: the data sits unencrypted and readable on the node, because that is what makes the high-performance ClickHouse queries and the data-lakehouse composition of this section possible—and the bet is that users do not care that the node operator can read their data, because they own the control over who *else* can reach it: a grant the user gives is a row the user deletes, and the operator is bound by law, not encryption, not to tamper with it. What the user owns is the policy, not the secrecy. End-to-end private messaging already exists (Signal, WhatsApp); web10 does not compete with it, and it is not trying to.

The consequence of the readable, shared store is the fork Figure 2 draws. A web10 node pools millions of users in one shared store, so discover, trending, and ranking are just queries the application writes against it—cross-author relational composition inside a single analytical engine, with no cross-database synchronization dependency. Solid cannot take that fork: each user's data lives in that user's own pod, so a cross-user query must pull the whole network, and it does not terminate at scale. "Who is trending?" is not a query Solid can run; it is a crawl. The costs of the shared table are real, and they are the day-to-day operating envelope of §8: version deduplication on every read, merge pressure, point queries paid to an engine built for scans, and shared resources between tenants.

4 THE BOUNDARY: YOUR QUERY, COMPILED BEFORE IT RUNS

Application-written SQL over a multi-tenant store needs a decision before the caller's relational operations run. The division of labor is the point: the application's developer writes the query and tunes it; the platform guarantees the boundary, not the performance. The API accepts a constrained read-only SQL surface—not arbitrary database access—and its boundary is a *wall, not a membrane*. The social application's feed code builds that query over service names— `posts`, `reactions`, `comments`, `profile`—and its SDK posts it with the reader's token (`{sql, token}` to `POST`

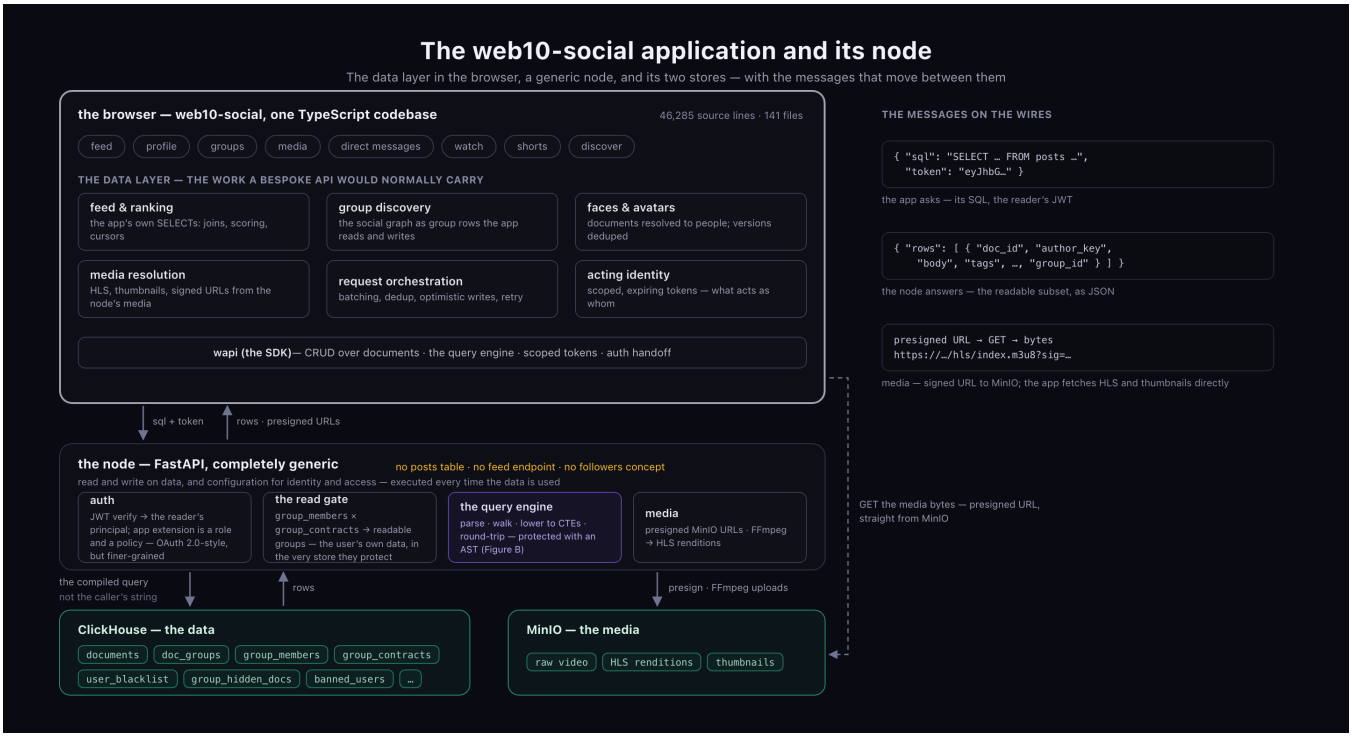


Figure 1: The web10-social application and its node—the data layer in the browser, the thin generic node, its two stores, and the messages that move between them.

`/v3/query`): the SDK adds no authority; the node decides what this reader may read.

The natural design is a membrane: wrap the caller's query and filter the *result*. It fails for one reason, and the reason is the whole section—the caller's query has already read the raw table inside the parentheses, and an outer filter cannot undo an aggregation that baked everyone's data in:

```
-- counts every user's comments; the outer group filter
     is too late
SELECT author_key, count(*)
FROM documents WHERE collection_name = 'comments'
GROUP BY author_key
```

The aggregate has collapsed all authors into counts before any row-level filter could run. Filtering the output of an aggregation is not access control.

web10 redirects the input instead. The node parses the caller's SQL to an AST and never sends that original string to the engine: it walks every table reference, rejects the raw node tables, the table functions, and the services the reader is not granted, and lowers each granted service name to a *boundary CTE*—the service's documents deduped to the latest not-deleted row, joined to the current `doc_groups` mappings, filtered to the reader's readable groups inlined from the read gate, with the block, sharing, hidden-document, and node-ban exclusions applied. The readable set of §2 is materialized *before* the caller's joins and aggregation run; an empty set yields an empty service. The query that executes is emitted from the validated AST, re-parsed on the way out and required to come back as exactly one statement—the caller's string is never executed, and Figure 3 follows one feed query through all of it.

The OLTP-era answer to many users on one database was row-level security inside the engine: PostgreSQL's row-security policies [11] evaluate a per-row policy expression while the engine runs the query. On a columnar store the boundary moves to the query, where the readable set is compiled in before any data moves. Both placements are sound; the cost of each is the matched workload §9 leaves to run.

5 EXTENDING THE PLATFORM COSTS A ROLE AND A POLICY

The social application is the node's killer app—the application compelling enough to pull users onto a new platform—and it is also the platform's hardest test, because it wants everything a bespoke backend would have given it. What it got instead is the division of labor of §4: its persistent data and authentication use the node's document, group, query, and media interfaces; its feeds are its own queries; its surfaces—text and discussion, image and mixed-media posts, vertical short video, long-form video, audio—are services it names over the shared store, composing on the same document, group, and query substrate rather than shipping a backend per surface. In the measured source snapshot, the application-local frontend is **46,285 physical source lines across 141 files**: 45,846 TypeScript/TSX lines and 439 CSS lines under `src/`, excluding tests; counts include comments and blank lines and exclude dependencies and tooling. Figure 4 shows the

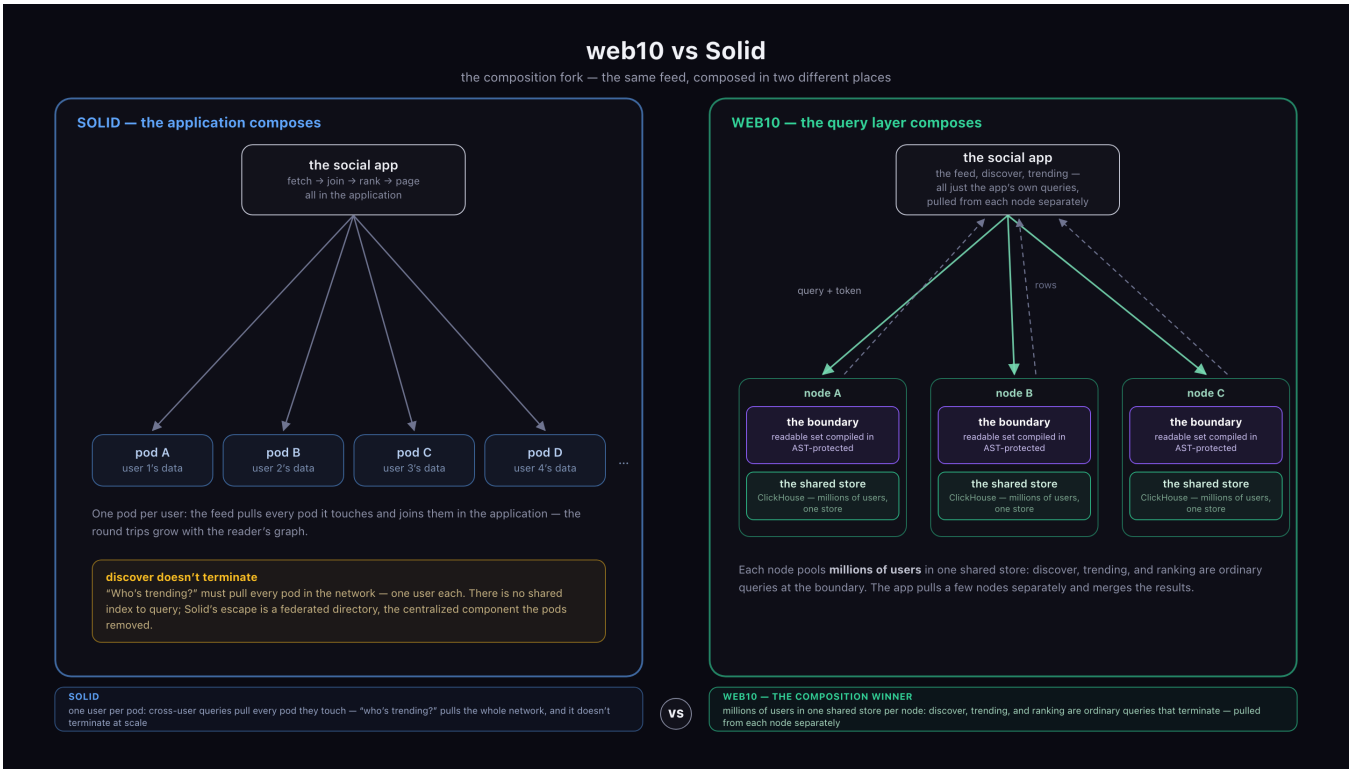


Figure 2: web10 vs Solid: where the composition happens—the same feed, composed in two different places.

social surface the feed composes over: a group—a shared space on the user's node—whose membership, roles, and per-post visibility are the application's choices over the generic grants of §2.

Extending the platform is as simple as a role and a policy: a new application gets an app contract, and the user grants it the per-service reads it needs. It is the same job OAuth 2.0 does for cross-platform data sharing [5], but the grant names the service, the group, and the operation—far more fine-grained than a scope. Two small static applications prove the seam: *Snake* associates completed runs with player identities; *Marketplace* associates listings with seller identities. Each writes its own service (`snake_scores`, `marketplace_listings`) and reads the social application's `profile` documents through the user's own grants. Neither modified the social application, and neither added a game, commerce, leaderboard, or seller-profile endpoint to the node. The common dependency is the document convention and the access primitives—not an application-specific API agreement:

```
WITH latest_profiles AS (
  SELECT author_key, body FROM profile
  QUALIFY row_number() OVER (
    PARTITION BY author_key
    ORDER BY updated_at DESC, created_at DESC,
             doc_id DESC) = 1
)
SELECT s.author_key,
       JSONExtractInt(s.body, 'score') AS score,
       p.body AS profile_body
FROM snake_scores s LEFT JOIN latest_profiles p
  ON s.author_key = p.author_key
ORDER BY score DESC LIMIT 50
```

Both service names resolve to reader-filtered relations before the join, so permission does not leak through it: an app-level grant on `profile` does not itself open a private profile group, and an ungranted service is rejected before execution. Figure 6 is the result in the browser—direct captures with synthetic account and listing data; the full-page, consent, and denial captures are retained in the artifact (§9.1); Figure 5 is why the fork matters.

On Mastodon and Bluesky, a second application reads through the platform's fixed endpoints (untunable) or replicates the user's records into its own database—Bluesky's own statusphere tutorial stands up a sync worker and a separate store just to have a feed: delegation is app-to-*user*-data, the object model stays fixed, and there is no shared store to compose across—a new read pattern is a new backend, and the model has no record shape for a hi-score or a listing. Here, a second app gets a typed read on the first app's data and joins it in SQL on one shared columnar store, provisioned on the node—no database to stand up: a new read pattern is a new query.

6 THE NETWORK: VERIFIED, NOT REGISTERED

The access model of §2 is defined for one node. This section states the model that makes it a network: the address, the handshake, and the domain.

The identifier is `provider/username`—the provider is the node, the username is the user, and the pair is globally unique across the federation. A user's home node is where their account and

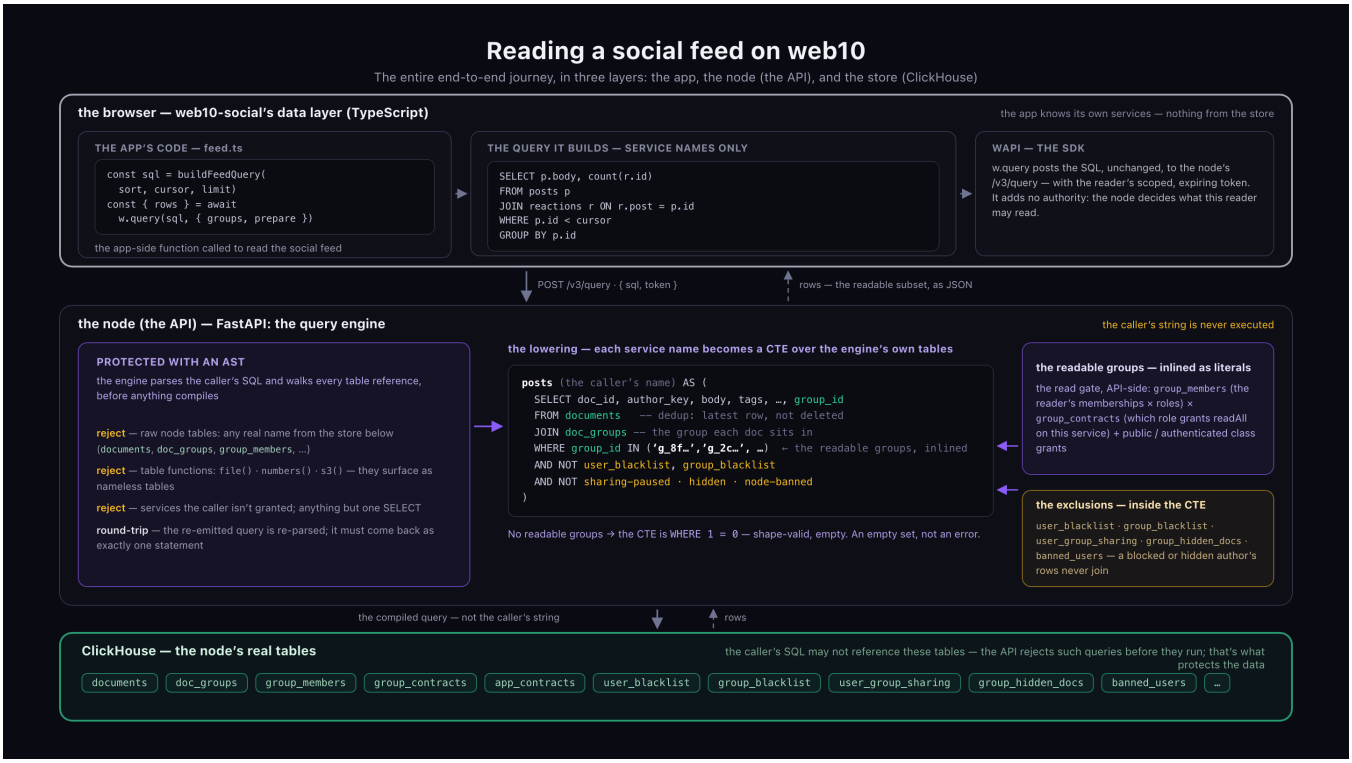


Figure 3: Reading a social feed on web10: the entire end-to-end journey, in three layers—the app, the node (the API), and the store (ClickHouse).

their grants live; a user's following set can span N nodes, and they hold exactly one account. When that user reads content on another node, they do not sign up there: their client presents their home-node token, and the foreign node obtains home-issuer certification, resolves the principal, and applies its own membership and grant checks. Identity is *verified, not registered.*

The read model is client-mediated. When the user's feed spans nodes, the *client* makes the cross-node calls: it holds the home-node token, groups the following set by provider, and queries only the distinct nodes the user follows people on—targeted, not broadcast. Each node returns its own access-gated result set; the client performs the ranked merge. A node never queries another node's store: the cross-node boundary is the HTTP API and a verified token, and a foreign node's response is untrusted content—never query input, never a key into the local store, never a grant. Database-level cross-server mechanisms (ClickHouse's `remote()`/`cluster()`) are for a trusted cluster under one operator; using them for federation would make a node operator's database credentials a federation credential.

`/certify` is the handshake that makes the verified-not- registered step cheap. Email has already solved the analogous problem for messages: an arriving message is asked *is this sender who they claim to be?* (SPF, DKIM), and that check grants the message no access— the recipient's own filters still apply. `/certify` asks the same of an arriving token: the serving node asks the home issuer, gets one cached answer per token, and that answer verifies and grants nothing— a handshake, not a registry, because every node still re-checks its own rules on every read. Same shape, different mechanism. Figure 7 draws the path for one read: node B authenticates the home-node token over the issuer's HTTPS `/certify`, then runs the same access check node A runs on local reads—contracts and membership, against B's own store, on every read.

The username need not be an opaque string, and this is where the audience stops belonging to the platform. Through the authenticator's claim flow, a user binds a DNS domain to their account: a TXT record at `web10.domain` carries `web10.v=1; provider=node;user=username`, and an authenticated `POST /v3/domains/verify` request makes the node perform the DNS lookup itself, checking that the record names this node and this account before storing the verified alias. The record is a pointer, not a credential: DNS lookup is discovery, not authentication, and the username remains the principal—groups, tokens, and the read gate never key on the domain. It is the same pattern the web already uses for identity claims (DKIM, SPF, `OpenPGPkey`), applied to a social identity.

The consequence is a *public* subscription that outlives the provider. The concrete case: a creator's audience follows the domain `jacobhoffman.xyz`, not the `provider/username` the domain currently points to—which may be one account today and another tomorrow. The follow is data the *follower* owns: a private document in the follower's own store whose body is exactly the

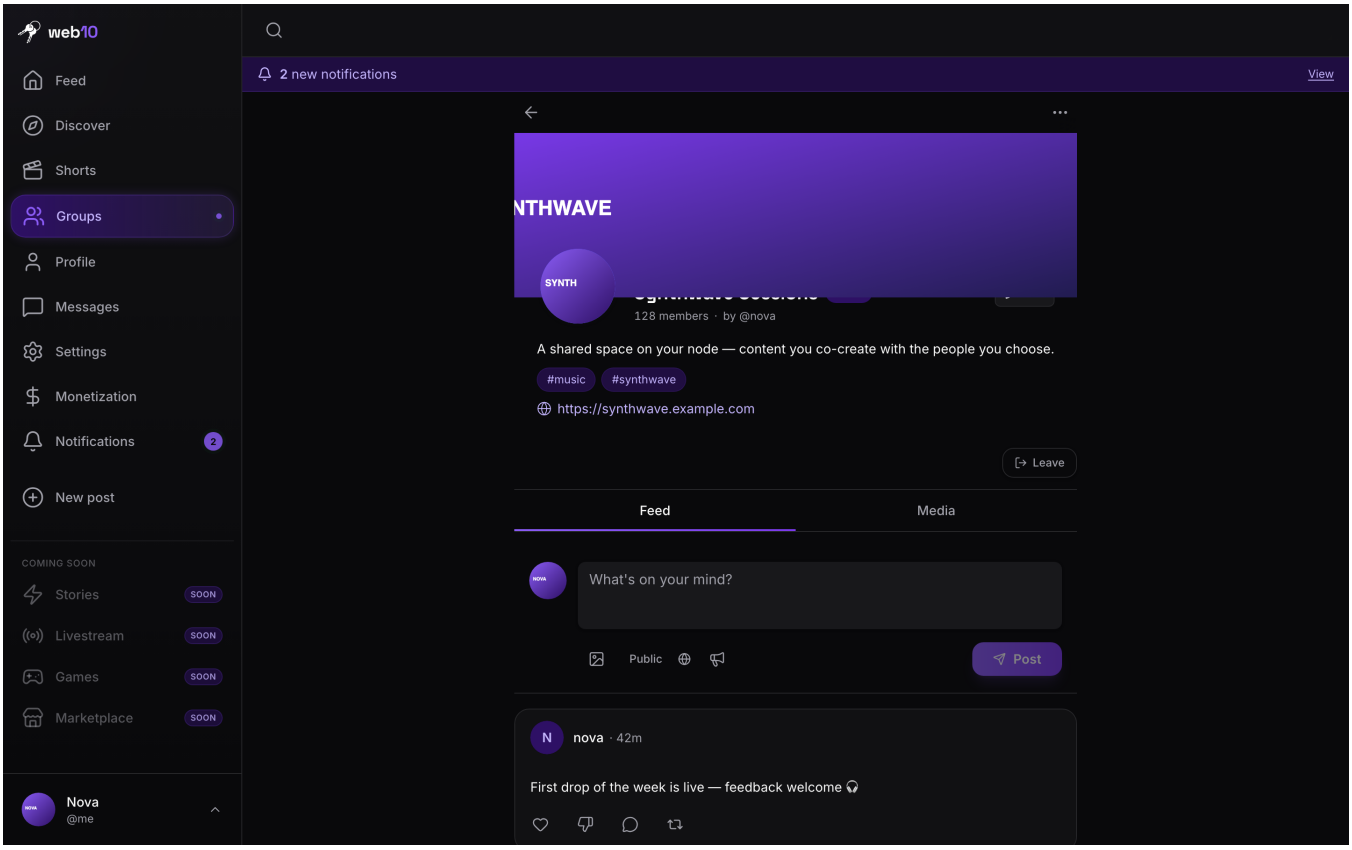


Figure 4: A group—"a shared space on your node."

domain, written once. When the creator moves—new account, new provider—the creator re-points the TXT record, and the follows travel with the domain: the follower's client re-resolves the address on each read, and no platform holds a ban switch over an address the user keeps. The node verifies the record by performing the lookup itself, but the record is untouchable to the node owner, who controls no part of it; a missing or ambiguous record gives an explicit unavailable state—the follow stays, and restoring the record recovers it without refollowing. Figure 8 draws the move.

7 THE MONEY: THE FREE WEB SHOULD HAVE ADS

Solid, Mastodon, and Bluesky ship no ad mechanism—ad-free by conviction, a documented design property of each (§10)— so on those platforms the creator's only revenue path is a cut the platform takes, and the platform's leverage over the creator is measurable: reach throttling, demonetization, deplatforming. web10 takes the opposite position: the free web should have ads, and the ad architecture matters. The position is on monetization, not on payment: the node processes no payment and takes no cut of the creator's revenue. It is on the rail that Figure 9 draws.

Every application that writes into the store inherits the rail: a new ad surface is a new query, not a new backend. web10-social builds the ad catalog today; the snake game and the marketplace inherit it—any of their documents can be a pinned ad target or a node ad. The node attaches the ads at the boundary, checked the same way every other read is checked (§4), with the density split explicit: the creator's layer is dense—the influencer's own offers, an affiliate link, a sponsorship, the node's creator tier, land on roughly 80–90% of their content; the node's layer is sparse—the operator's overlay lands on roughly 5–10% of posts, node-controlled, default 10, deterministic per reader, round-robin across the inventory. Both can be present on the same post, and the node's overlay never suppresses the creator's monetization. The inventory is tagged documents plus an offer object—kind, the link that pays, call to action, disclosure—that the platform never rewrites and never cloaks.

The money itself stays off-platform. The creator keeps 100% of affiliate and sponsor revenue: the link pays the creator, the platform takes no cut and processes no payment. The node operator is a media company: the overlay sells the node's own attention directly, and the operator keeps the bulk of it—roughly 85–90%. And because the overlay density is node-controlled, nodes compete for the influencers by lowering their rates. The horizon is a separate layer, not this system's claim: an ad exchange, a payments rail, and the creator's own layout sit ahead of it.

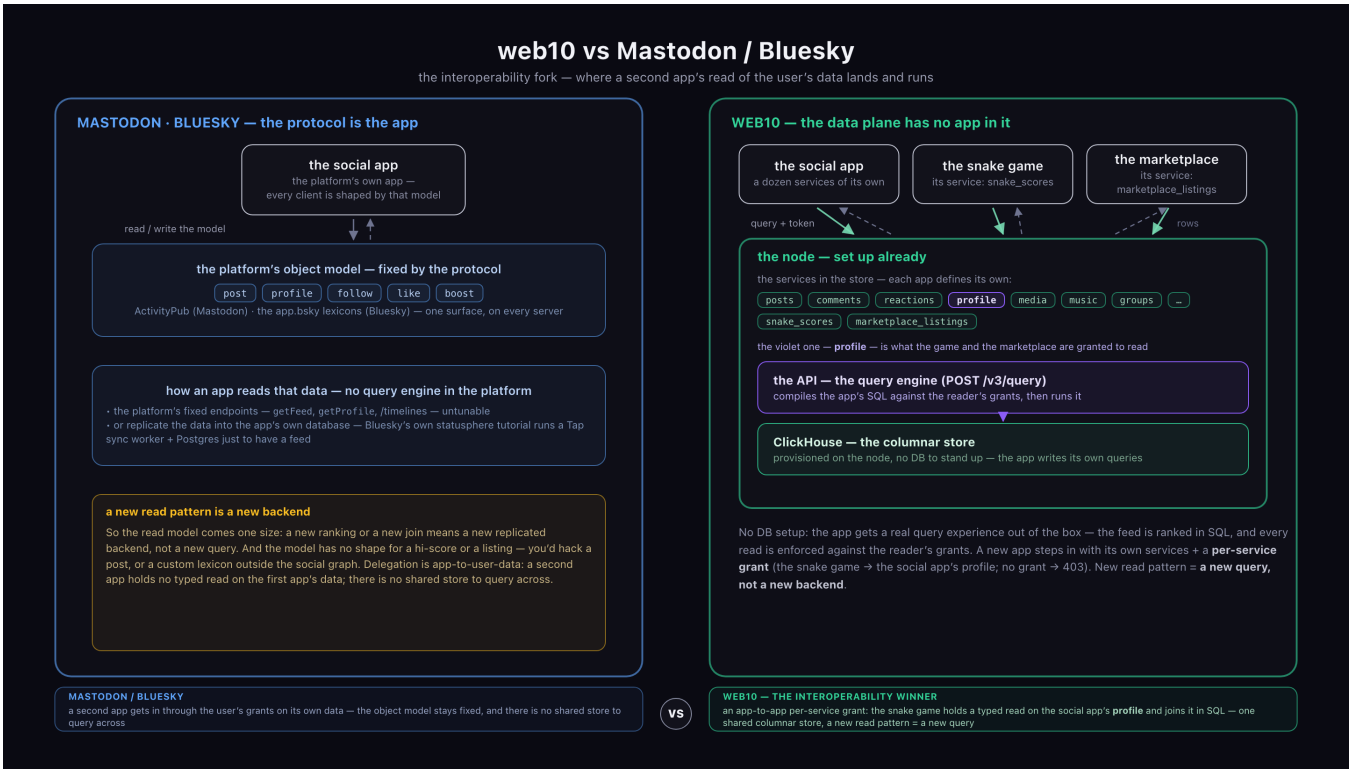
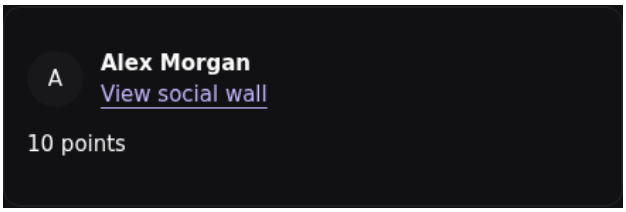


Figure 5: web10 vs Mastodon / Bluesky: the interoperability fork—where a second app's read of the user's data lands and runs.

Snake: earned score + social identity



Marketplace: listing + social identity

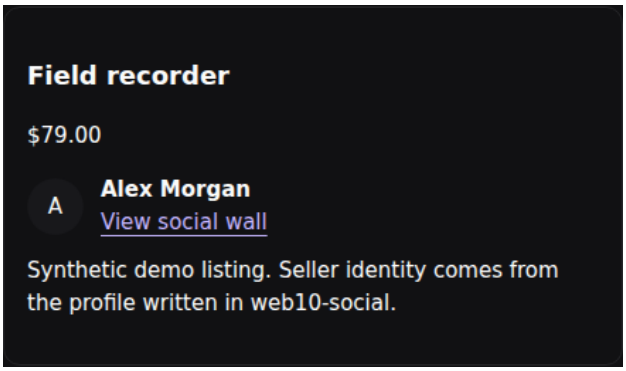


Figure 6: One social-written profile, two application views: Snake's earned score, Marketplace's listing.

The market the rail makes is Figure 10: a 6×6 matching market, six nodes pricing their own terms against six influencers carrying their own market properties. The nodes: one bans the least, at the higher price; one is a certain political match; one is the lowest price with the heavy overlay; one cuts the overlay to 5%; one is community-governed moderation; and the self-hosted floor—free, never gated—the outside option that keeps the prices honest. The edges carry the dynamics: an influencer flips when the preference changes, flips when the node *acts*, shops the terms fit—and a node loses two of its three when price alone does not hold the audience. Because the follow is the fan's own data, bound to the domain (§6), an influencer changing nodes is a rematch, not a loss. Because the overlay density is node-controlled, the market between influencers and nodes is a matching market on *terms*—price, bans, overlay, moderation style. The three rents the paved platforms hold over the creator—reach throttling, demonetization, deplatforming; today's rent measured at one million subscribers, three hundred thousand delivered—are structurally out. The competition is on terms, not on leverage.

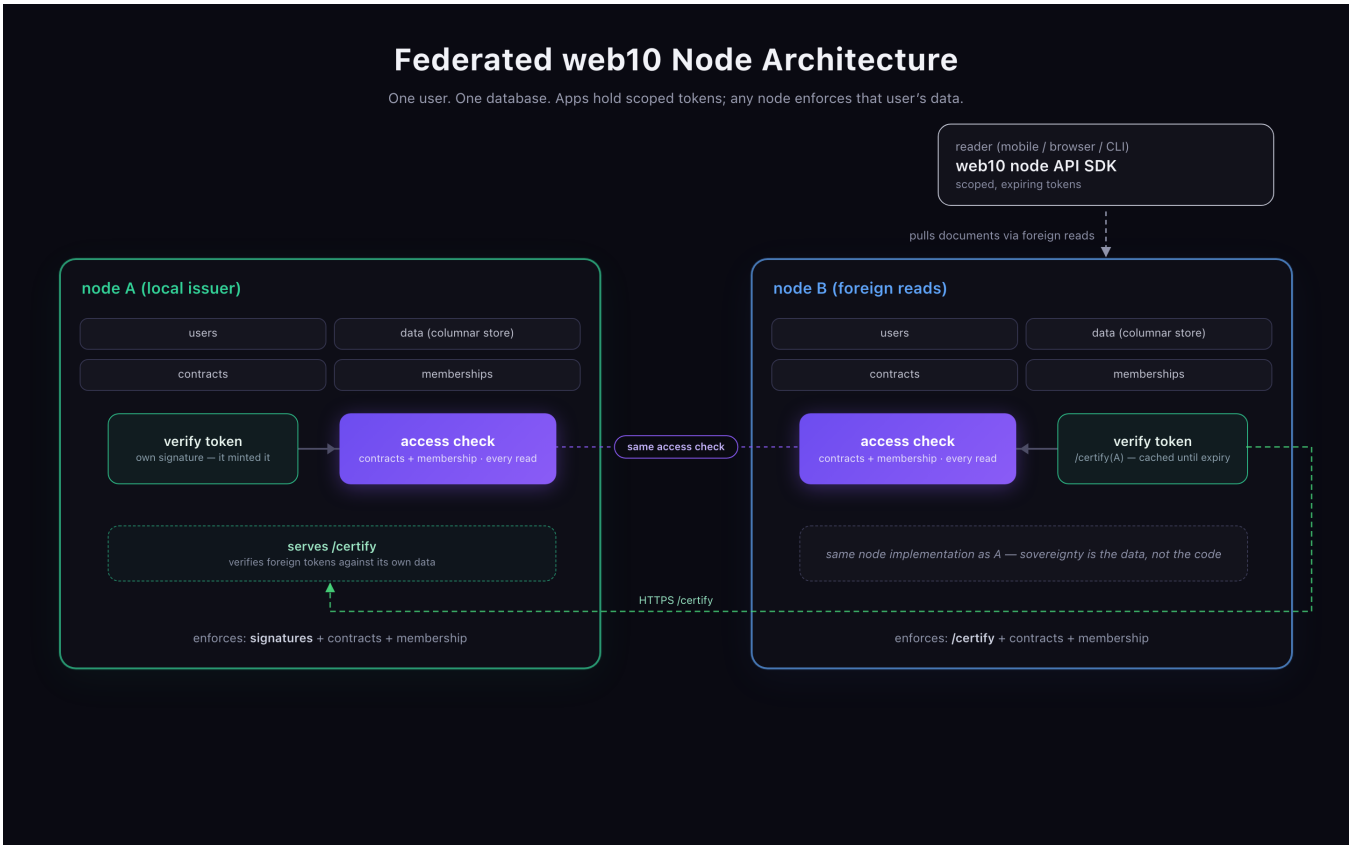


Figure 7: Pulling the web10 feed across nodes: one node implementation, two sovereign stores; remote data is content, never control.

8 LESSONS FROM RUNNING IT

The industry track asks what running a system taught that designing it did not. Three things, in roughly the order they bit.

8.1 The node keeps resisting the killer app

The platform's hardest dependency is its own killer app, and the killer app always asks for bespoke surface: a feed is faster to ship as an endpoint than as a client query; a group's name, description, and banner photo look like exactly the kind of thing a `group_identity` table would hold; a session oracle that knows about the followers group would make the social app's re-authentication path trivial. Four such node surfaces were built, and all four were removed: the dedicated feed endpoint (replaced by an application-written query over the services it composes), the group-identity table with its own endpoint (replaced by documents in an app-named service read through the normal CRUD and grant path), the followers-group check inside the generic session oracle (moved to the one application that knows what a followers group is), and an ad path that hard-coded a collection name (replaced with a tag and group membership the applications themselves manage). The test that caught each one is one sentence: *would a notes app, a music app, or a shop use this endpoint?* The cost of holding the line is real and visible—the client does more work—and the payoff is §5: a game and a marketplace with no relationship to the social application's code, reading its profiles through the same boundary, no node change when either appeared.

8.2 A self-healing schema raced its own readers

On a fresh stack, the node self-heals its metadata schema with a deferred `ALTER TABLE` at boot, adding columns to `group_contracts` that older databases lack. An early read that lands before the `ALTER` commits fails with an unknown-identifier error, and it is easiest to hit exactly where the stack is newest: a clean database, a cold boot, the first request. The lesson: the schema additions must land synchronously and be guaranteed before the first read of that table, or the columns must exist at table creation—and the regression that guards it boots a fresh database and immediately reads the table. The general lesson is the one that survives the particular bug: a self-heal that may lag is fine; a self-heal that may *race* is not. Test the fresh-boot path, not just the steady state.

8.3 What the shared columnar table costs, day to day

The data-plane choice of §3 trades physical isolation for cross-author composition, and the running platform meets the price

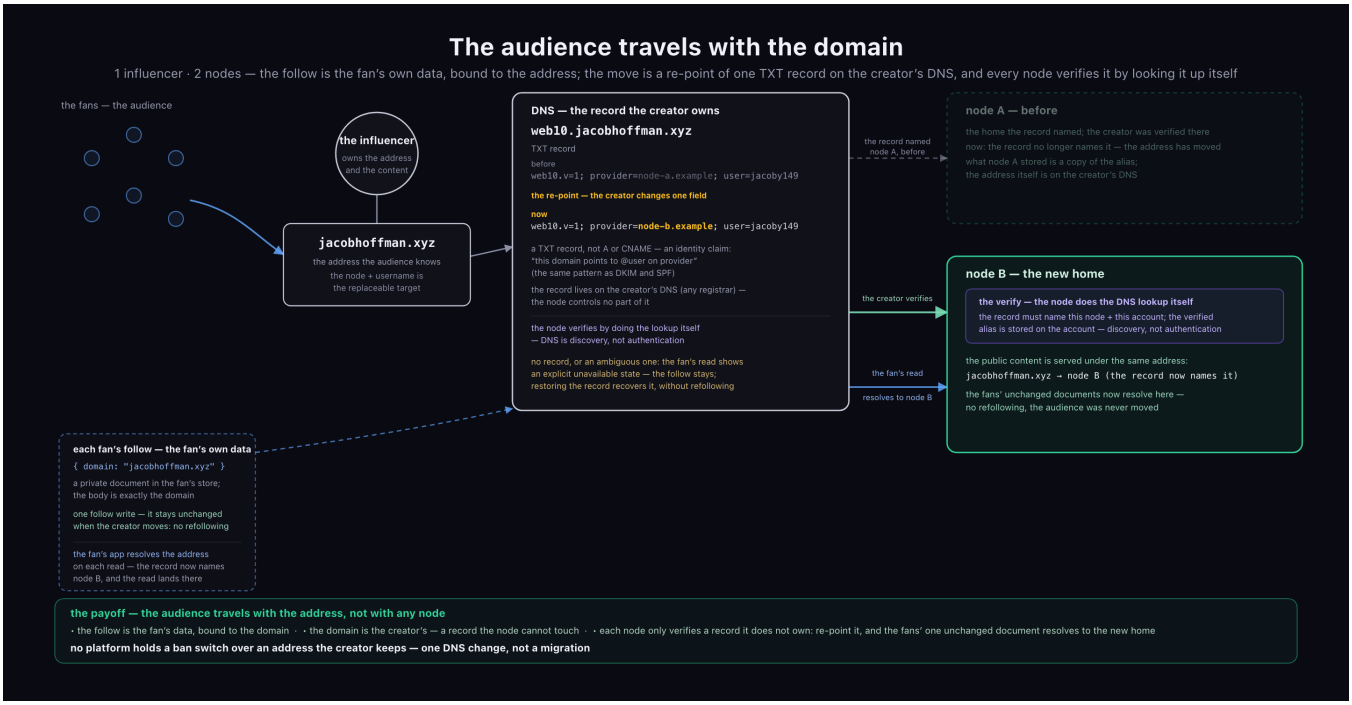


Figure 8: The audience travels with the domain: one influencer, two nodes, the DNS flip—one DNS change, not a migration.

of that trade every day. Every read pays the version deduplication, because `ReplacingMergeTree` settles versions only when it merges. "Deletion" is an insert, so a tombstone is a row that must be ranked, not discarded. Merge pressure is background CPU that does not show up in request latency until the host is busy. Point queries are paid to an engine built for scans. And every tenant in the node shares the same resources. These are not defects of the design; they are the design's operating envelope—and they are the reason reader isolation lives at the query boundary of §4, not in a physical partition.

9 THE EVIDENCE

The evidence is connected, bounded, and named; each part is organized as the question it answers. Does an unrelated application actually reuse another's data through the access boundary, with consent and revocation exercised in a real browser (§9.1)? Does a grant reach across the node boundary, and can the owner revoke it locally (§9.2)? Does a public subscription outlive its provider (§9.3)? What does the access layer cost (§9.4)? And is anyone actually using this (§9.5)? The first four are synthetic data on isolated test stacks; the operating point is a read-only observation of the production node. None is a capacity or scale result.

9.1 Does an unrelated application actually reuse another's data?

A fresh account logged into web10-social through its authenticator popup and edited its profile name to Alex Morgan. The browser approved Snake's app request; the player earned and saved ten points, and the leaderboard displayed the joined profile. After app-contract revocation, a new query returned 403 and the leaderboard cleared the result and displayed an access error. After sign-out, Marketplace requested its service permission through the popup, published a listing, and displayed the same profile; reloading the listing's detail URL preserved the result. API assertions checked exact authors, row counts, profile bodies, and listing identity, so the username fallback alone could not satisfy the test.

A separate HTTP case with three accounts and two social-written profiles checked the exact two-author joins: with no app permission for `profile` both queries were denied; granting it enabled the joins; revoking the app contract denied them again. A separately labeled private-profile fixture isolated the group gate: membership enabled an exact inner join; removal emptied it while the scores remained readable.

All three cases—desktop at 1440 pixels, mobile at 375 pixels, and HTTP correctness—passed in the final fresh-stack run on 2 November 2026, with zero retries or skips. The run used a four-vCPU, 8-GB DigitalOcean sfo2 host with ClickHouse 24.8.14.39, Playwright 1.63.0, and Chromium 153; the exact source, configuration, image identities, results, and all 16 original captures are retained in the interoperability artifact. The hosted apps share one browser origin, although the HTTP case supplies a separate app Origin: these controls establish named grant/revocation behavior across applications, not separate-origin isolation or an independently recruited developer study, and they establish behavior, not a stale-access or capability-expiry bound.

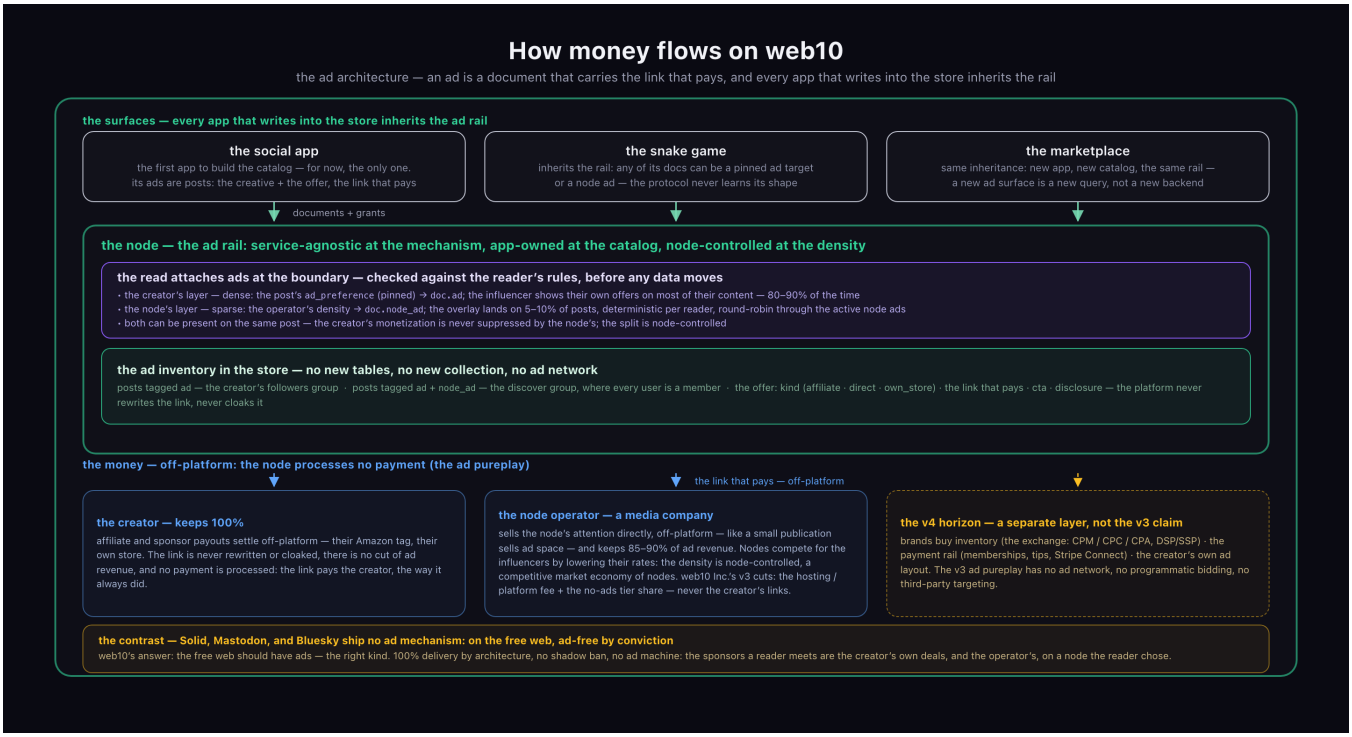


Figure 9: How money flows on web10: the ad rail—the creator keeps 100% of their monetization, the node operator is a media company, the platform takes no cut.

9.2 Does a grant reach across the node boundary—and can the owner revoke it locally?

Two API processes, separate ClickHouse and MinIO instances, independently generated HS256 signing secrets, and real HTTPS home-issuer certification with an ephemeral test CA. Bob creates a private group and document through B's API. Alice's A-issued JWT initially fails both B's document read and B's query—and giving a *distinct B-side Alice* the membership also fails to authorize A's Alice: two Alices, same name, told apart only through the qualified principal. Bob then grants the exact foreign principal through the owner API; both read paths return the exact document; removing that membership denies both; regrant restores both.

All steps reused the same JWT, and the certification traffic shows why the model works the way §6 states it: A's TLS proxy records exactly *one* successful certification across the entire lifecycle, while B rechecks access on every read. A foreign-provider JWT signed with B's key is rejected twice, with one failed issuer call and a negatively cached second rejection; an independent Origin-bearing query gate denies a local namesake's app grant, allows a qualified foreign fixture, and denies again after its revocation. The scenario completed without retries on an unmodified source snapshot; exact source, configuration, request outcomes, image identities, and cleanup records are retained in the experiment artifact. This is a correctness experiment, not a deployment or load trial: the test CA and Docker-internal addresses do not establish public-internet interoperability, and owner grant/removal is not foreign self-join.

9.3 Can a public subscription outlive the provider that hosts it?

A separate research client stores only the creator's normalized domain in a private app document at the reader's independent home. It freshly resolves the existing TXT pointer, checks the selected account's verified alias, and reads public posts anonymously—through ordinary CRUD and groups and the existing authenticated domain-verification endpoint, not a new node follow API.

On desktop (1440×1000) and mobile (375×812), the creator logged in, published the TXT record, and verified the old account through actual local UDP DNS; the reader clicked Follow once. Then the old API and TLS services were stopped. With the original provider unavailable, the creator re-pointed DNS and verified a new account without the old provider's approval. Reload and ordinary reader login displayed new public posts with the identical stored follow ID and body, and one total follow creation; no old content was migrated. And the asymmetry that should hold: explicit old private access returned 200 while new-target private access remained 403—the public subscription follows the domain, but private access is not inherited by the new target. Figure 8 draws the move; the exact panel captures, the authenticated hookup, and the 375px evidence remain in the artifact.

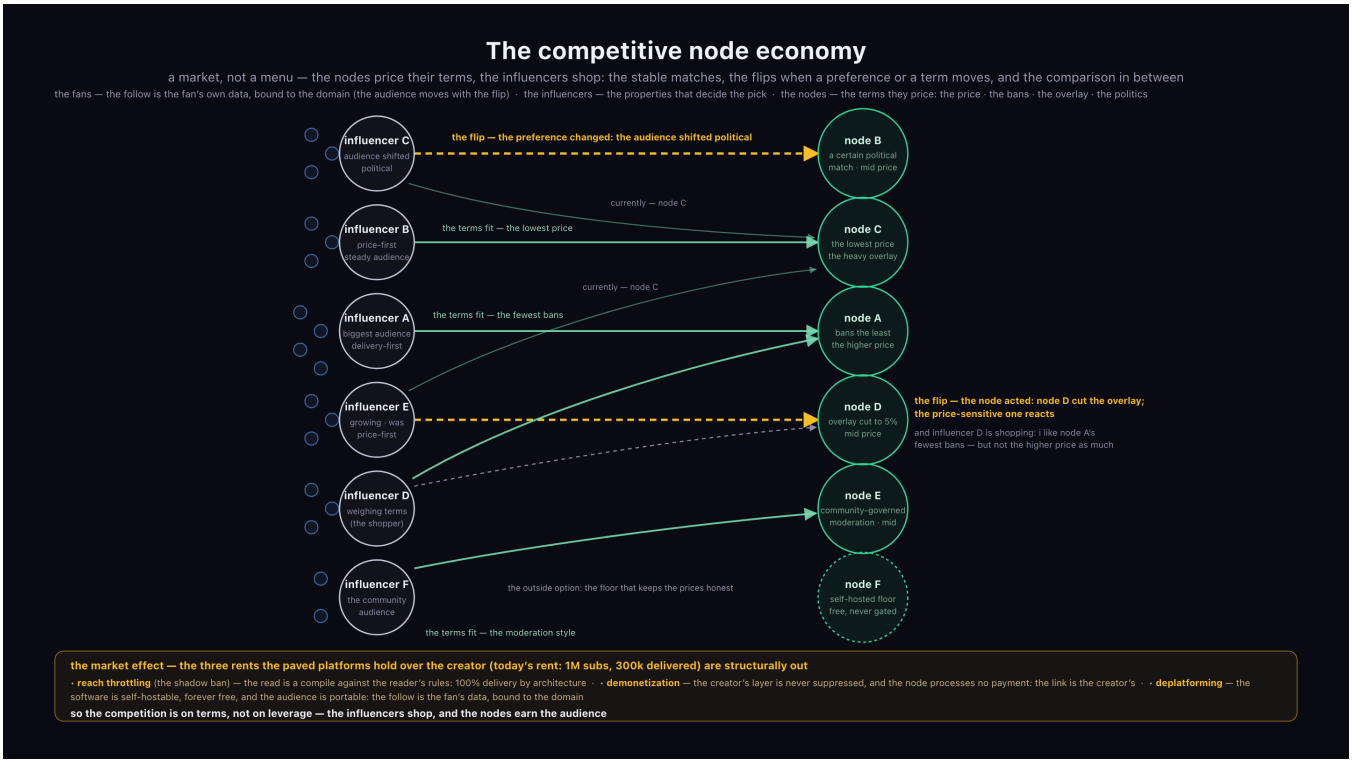


Figure 10: The competitive node economy: a 6 × 6 matching market—the competition is on terms, not on leverage.

Controls delayed real responses at five identity-transition boundaries and verified that logout or account changes prevented stale UI updates; missing or ambiguous DNS cleared displayed content without deleting the saved subscription; and the DNS/alias/public requests carried no credentials. Exact source and configuration, ten captures, request outcomes, and cleanup records are retained in the prototype artifact. This is a configured two-node client, not deployed social follows or private principal continuity: domain control, resolver availability, and the availability of the reader's home and new provider are assumptions rather than guarantees; domain sale, theft, and expiry are not solved; and the result demonstrates bounded public subscription continuity, not absolute unbannability.

9.4 What does the access layer cost, and where does the cost live?

The read path resolves the reader's readable groups before the query executes. We measured that cost to understand the design, not to report a latency distribution: the question was whether per-group metadata resolution scales with the number of candidate groups, and what batched resolution buys. Figure 11 plots the trial; the numbers it carries are stated here, with the scope that bounds them.

Setup. One sequential trial on a DigitalOcean sfo2 `s-4vcpu-8gb` host (four vCPUs, 8 GB nominal RAM, Ubuntu 24.04, Linux 6.8, Docker 29.1.3), co-locating a single Uvicorn process and ClickHouse 24.3.18.7. The synthetic seed (seed 42) created 1,000 creators with one small post each and a reader who joined all their groups; those groups also granted public reads. Personal queries supplied $K = 10$, 100, or 1,000 candidate groups on this same dataset and host; a separate discover query supplied one board group ($K = 1$), containing ten additional posts. Twenty samples per cell, 400 measurements; all 320 HTTP requests returned 200. The queries selected document IDs and bodies, ordered by creation time and ID, with a limit of 20; they did not execute engagement joins, media preparation, or browser page assembly, and they supplied no Origin header, so this trial does not exercise the browser-origin app-contract gate.

Variants. The per-group resolver performs point metadata queries for each candidate group. The batched resolver fetches memberships, principal-class grants, and role definitions across the candidate set in three database queries, then resolves access in memory. We measured each resolver with real document execution (B) and a fixed 20-row synthetic scan result (C); C retains the policy/compilation path but is not an output-equivalent execution. A third variant (A) precompiles the known-readable projection and times ClickHouse execution directly, outside the HTTP handler.

Finding. The per-group resolver's cost grows linearly with the number of candidate groups: increasing K from 10 to 1,000 increased median readable time 100.44× (75.03 ms to 7,535.71 ms; p95 93.30 ms to 8,504.61 ms), against 9.12× for the batched resolver (23.47 ms to 213.98 ms; p95 31.17 ms to 284.03 ms). At $K = 1000$ the batched path is 35.22× faster on readable time and 21.04× faster

on handler time (363.31 ms median handler). Stubbing the execution keeps the separation: 8,388.39 ms per-group versus 222.57 ms batched at $K = 1{,}000$. The per-group path still completes requests, but its 7.64-second median at $K = 1000$ is a practical bottleneck for a reader who follows many creators; the batched path's 363 ms is not. The direct-scan baseline stayed at 28.6–54.2 ms across $K = 10$–$1{,}000$: the document scan was already done while the access layer was still resolving. This is the design decision the measurement drives: the query endpoint should use the batched resolver for large candidate sets.

The tradeoff is honest in both directions. Batching is not uniformly cheaper: at the discover board's $K = 1$, the per-group path is faster (9.53 ms readable versus 25.37 ms batched), because three fixed round trips dominate when there is one group to check. And batching does not make the cost constant: the batched handler median still grew 4.47× over the 100-fold increase in K, as the metadata to process and the compiled query to emit both grow with the candidate set. The trial is a single sequential run on a simplified projection; it characterizes the access layer's cost structure—metadata fan-out is the dominant term at large K, and batching is the mitigation—and does not establish a latency distribution, a capacity bound, or a causal authorization-overhead estimate; $B - A$ is a pipeline-versus-scan comparison, not a matched causal measurement. Exact source, configuration, and per-sample measurements are retained in the measurement artifact; an earlier 100-creator trial on a different host showed the same qualitative tradeoff and is not pooled with this trial.

9.5 Is anyone actually using this?

The platform is deployed and serving authenticated users. A read-only observation of the production node on 28 October 2026 recorded 597 account rows, 593 live documents across 30 distinct authors and 10 services, 49 group contracts, and 1,298 membership rows, on a host running the social, authenticator, and API frontends. Nine authenticated browser navigations (following feed, music, groups) returned HTTP 200 with a retained session and zero uncaught page errors; the following feed rendered 20 posts in a median 5,396 ms of full-page readiness. This is an operating point—evidence the system runs and serves real users—not a capacity or scale result: the host also runs a development stack, the observation is a snapshot without an activity window, and the readiness figure includes frontend startup and network work, not server-query duration. The 5.4-second figure is the kind of number the industry track exists to report: it is real, it is bad enough to matter, and the full-feed dissection named in §11 is the measurement it asks for.

10 NO NEW PART, ONE COMBINATION

The contribution is an integration of established ideas: user-addressed storage, user-managed grants, and application-defined queries over authorized relations. The two comparison figures of §3 and §5 organize the matchup visually; what follows is positioning, not a survey. We discuss design differences from the cited systems rather than claiming that no prior implementation combines these capabilities.

Solid: the closest prior work. Solid [13] separates applications from personal data stores (pods), with social application work [12] and a maintained Community Solid Server [6]. WAC supports resource permissions and group agents through `acl:agentGroup` [4], and ACP adds agent, client, and issuer matching with allow/deny mode sets [2]—web10 does not claim Solid lacks group or client authorization; it has both. The distinction is in the *shape* of the model: Solid's access control is resource-centric—ACLs attached to linked-data resources, evaluated per resource—while web10's is service-centric: a uniform principal-class × service × operation vocabulary in which the grant is a row naming a globally addressable principal, exposed to the end user rather than an infrastructure operator. And the query surface: Solid's cross-pod query cost depends on deployment, caching, and linked-data traversal; web10 composes application-defined SQL over a shared columnar store with the access decision made on the input (§4)—the fork of Figure 2. A performance comparison would require a matched workload and deployment; we do not claim one here.

Federation: the content moved, the access layer did not. ActivityPub [9] (Mastodon) defines federated server/server activity delivery; its authorization is platform-defined—follow/block and collection-level visibility—not per-principal grants over application-defined services. ATProto [8] (Bluesky) separates repositories, relays, and AppViews, supporting user agency and provider switching; its delegation is app-to-user-data—per-collection, per-action permissions over the user's own repo—and its read path is the platform's fixed endpoints or a record the app has replicated into its own database. Neither has a shared store to compose across, and neither lets a second application hold a typed read on the first application's data. web10's seam is the one Figure 5 draws: service-scoped, user-managed grants, and application-defined queries over the resulting authorized relations—the seam §5 exercises with two applications that share nothing but the document convention.

The columnar boundary: the placement the OLTP answer did not make. The tenant-isolation literature—shared tables with an organization identifier (Force.com [15]), schema mapping [1], the shared-versus-separate spectrum [7]—is the closest systems ancestor, with the tenant at the organization; web10's key-space per user takes that spectrum to its limit (§3). PostgreSQL row-security policies [11] are the alternative enforcement location: per-row policy evaluation inside the engine. At industrial scale, Zanzibar [10] is the closest prior art in relationship-based authorization: a uniform ACL data model across hundreds of services, with consistency as a first-class problem—but the model is infrastructure-side: the operator defines the relationship schema and the service enforces it. web10 exposes service-scoped grants as user-managed data, names globally addressable principals, and enforces access at the query layer over a shared columnar store. TAO [3] is the deployed precedent for read-time checking at social-graph scale (a billion reads per second, the privacy check per item, pull not push): web10's boundary is the same idea—every row access-checked on its way out—applied to service-namespaced documents in a columnar store, with the check expressed as a reader-filtered CTE compiled in before the caller's aggregation runs, rather than a per-item

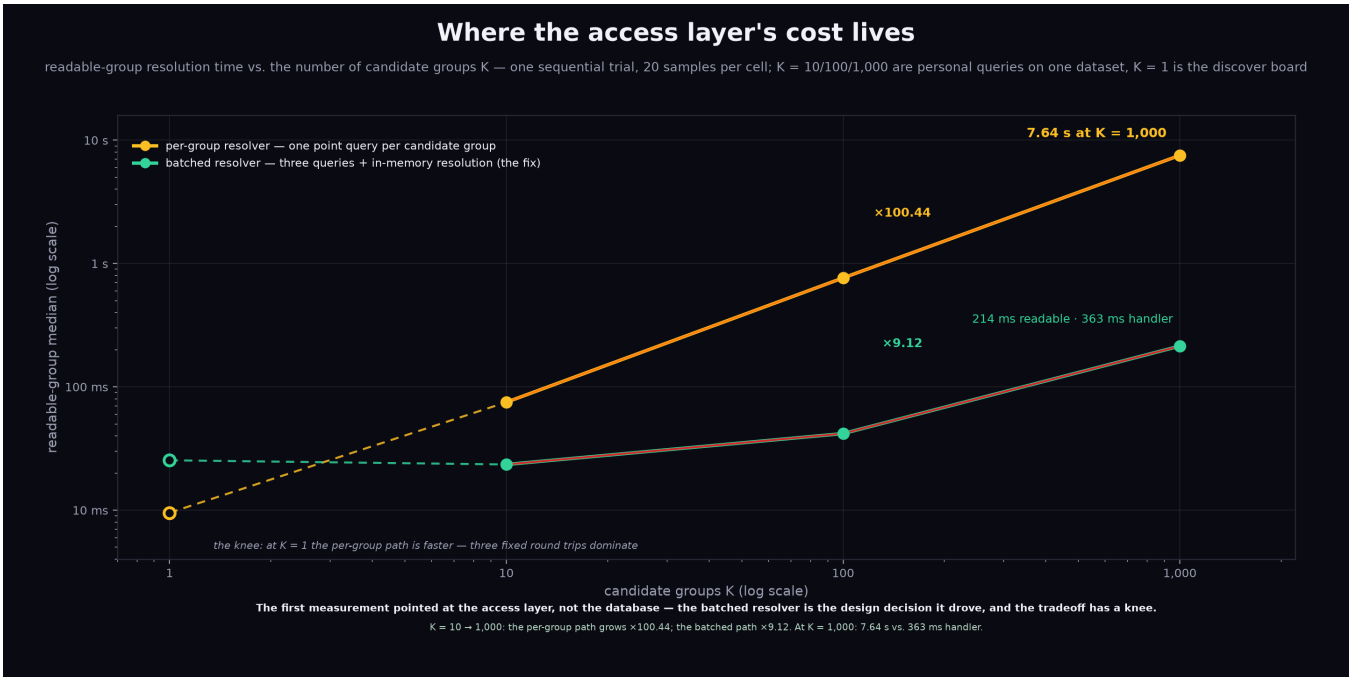


Figure 11: Where the access layer's cost lives: readable-group resolution time versus the number of candidate groups K (log-log)—the trial of §9.4.

predicate. The matched policy-and-workload comparison of these placements is the forward scope §9.4 leaves to run.

Monetization: the gap the free-web camp shipped deliberately. Solid, Mastodon, and Bluesky ship no ad mechanism. That is a documented design property of each—the free-web camp—not an oversight, and on those platforms the creator's revenue path is the platform's cut, with the rent measurable: reach throttling, demonetization, deplatforming. web10's answer is the ad rail of §7: the creator keeps 100% of affiliate and sponsor revenue, the node operator is a media company that sells the node's own attention, and the platform takes no cut and processes no payment—and the market the rail makes is Figure 10, where the competition is on terms, not on leverage.

11 CONCLUSION

What makes your data worth owning is the graph around it: who follows you, who belongs where, who may read what. No platform lets you keep that graph—the content is portable in principle, and the permissions are the part that stays. web10 is the generalization that changes it: a user-addressed network in which applications define their own document services over a shared columnar store in each sovereign node, in which a uniform, user-managed access model lets any principal on any node hold a role in a user's groups, and in which the query layer supplies the reader's service relations before joins and aggregation execute.

The evidence is connected, bounded, and named. A game and a marketplace reuse the social application's profiles with no node change and no API agreement, with browser consent and cross-author controls exercised end to end; a two-node experiment shows a foreign principal granted, reading, revoked, and restored under a single cached certification; a browser prototype keeps a public subscription alive after its provider goes dark—while the private grants correctly stay behind. And the metadata fan-out measurement identifies where the access layer's cost actually lives at scale: at 1,000 candidate groups, batching reduced median handler time from 7.64 s to 363 ms, while costing more at one group. This is a design decision, not a scalability claim—and it is the kind of concrete, measured tradeoff an operating system earns.

The generic core was earned, not designed from the top: the four built-and-deleted node surfaces of §2 are the visible part. And the market the data plane makes is why the reach is worth it: the creator keeps their own monetization, and the nodes compete for the creators on terms, not on leverage.

The forward scope is specific: federation beyond the isolated correctness slice (§9.2), full-feed and concurrent workload measurements, a matched shared-table policy baseline (§9.4), and revocation-latency bounds. The architectural contribution is the integration itself.

Email let the data leave the building. web10 lets the permissions leave it too.

REFERENCES

[1] Stefan Aulbach, Torsten Grust, Dean Jacobs, Alfons Kemper, and Jan Rittinger. 2008. Multi-Tenant Databases for Software as a Service: Schema-Mapping Techniques. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data (SIGMOD '08)*. ACM, 1195–1206.

[2] Matthieu Bosquet. 2022. Access Control Policy (ACP). W3C Solid Community Group, Editor's Draft. https://solid.github.io/authorization-panel/acp-specification/

[3] Nathan Bronson, Zach Amsden, George Cabrera, Prasad Chakka, Peter Dimov, Hui Ding, Jack Ferris, Anthony Giardullo, Sachin Kulkarni, Harry C. Li, Mark Marchukov, Dmitri Petrov, Lovro Puzar, Yee Jiun Song, and Venkateshwaran Venkataramani. 2013. TAO: Facebook's Distributed Data Store for the Social Graph. In *Proceedings of the 2013 USENIX Annual Technical Conference (USENIX ATC '13)*. USENIX Association, 49–60.

[4] Sarven Capadisli. 2022. Web Access Control. W3C Solid Community Group, Version 1.0.0-cr-1. https://solidproject.org/TR/wac

[5] Dick Hardt. 2012. The OAuth 2.0 Authorization Framework. IETF RFC 6749, Standards Track. https://doi.org/10.17487/RFC6749

[6] Joachim Van Herwegen and Ruben Verborgh. 2024. The Community Solid Server: Supporting Research and Development in an Evolving Ecosystem. *Semantic Web* 15, 6 (2024), 2597–2611. https://doi.org/10.3233/SW-243726

[7] Dean Jacobs and Stefan Aulbach. 2007. Ruminations on Multi-Tenant Databases. In *Proceedings of BTW 2007*. 514–521.

[8] Martin Kleppmann, Paul Frazee, Jake Gold, Jay Graber, Daniel Holmgren, Devin Ivy, Jeromy Johnson, Bryan Newbold, and Jaz Volpert. 2024. Bluesky and the AT Protocol: Usable Decentralized Social Media. In *Proceedings of the ACM CoNEXT-2024 Workshop on the Decentralization of the Internet (DIN '24)*. ACM, 9. https://doi.org/10.1145/3694809.3700740 Extended version: arXiv:2402.03239.

[9] Christine Lemmer-Webber, Jessica Tallon, Erin Shepherd, Amy Guy, and Evan Prodromou. 2018. ActivityPub. W3C Recommendation. https://www.w3.org/TR/activitypub/ Editors: Christine Lemmer-Webber and Jessica Tallon.

[10] Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, Jeffrey L. Korn, Abhishek Parmar, Christina D. Richards, and Mengzhi Wang. 2019. Zanzibar: Google's Consistent, Global Authorization System. In *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC '19)*. USENIX Association. https://www.usenix.org/system/files/atc19-pang.pdf

[11] PostgreSQL Global Development Group. n.d. PostgreSQL Documentation: Row Security Policies. PostgreSQL documentation. https://www.postgresql.org/docs/current/ddl-rowsecurity.html Accessed October 2026.

[12] Andrei Sambra, Amy Guy, Sarven Capadisli, and Nicola Greco. 2016. Building Decentralized Applications for the Social Web. In *Proceedings of the 25th International Conference Companion on World Wide Web (WWW '16 Companion)*. ACM, 1033–1034.

[13] Andrei Vlad Sambra, Essam Mansour, Sandro Hawke, Maged Zereba, Nicola Greco, Abdurrahman Ghanem, Dmitri Zagidulin, Ashraf Aboulnaga, and Tim Berners-Lee. 2016. *Solid: A Platform for Decentralized Social Applications Based on Linked Data*. Technical Report. MIT CSAIL and Qatar Computing Research Institute. http://emansour.com/research/lusail/solid_protocols.pdf

[14] Robert Schulze, Tom Schreiber, Ilya Yatsishin, Ryadh Dahimene, and Alexey Milovidov. 2024. ClickHouse: Lightning Fast Analytics for Everyone. *Proceedings of the VLDB Endowment* 17, 12 (2024), 3731–3744. https://doi.org/10.14778/3685800.3685802

[15] Craig D. Weissman and Steve Bobrowski. 2009. The Design of the Force.com Multitenant Internet Application Development Platform. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data (SIGMOD '09)*. ACM, 889–896. https://doi.org/10.1145/1559845.1559942